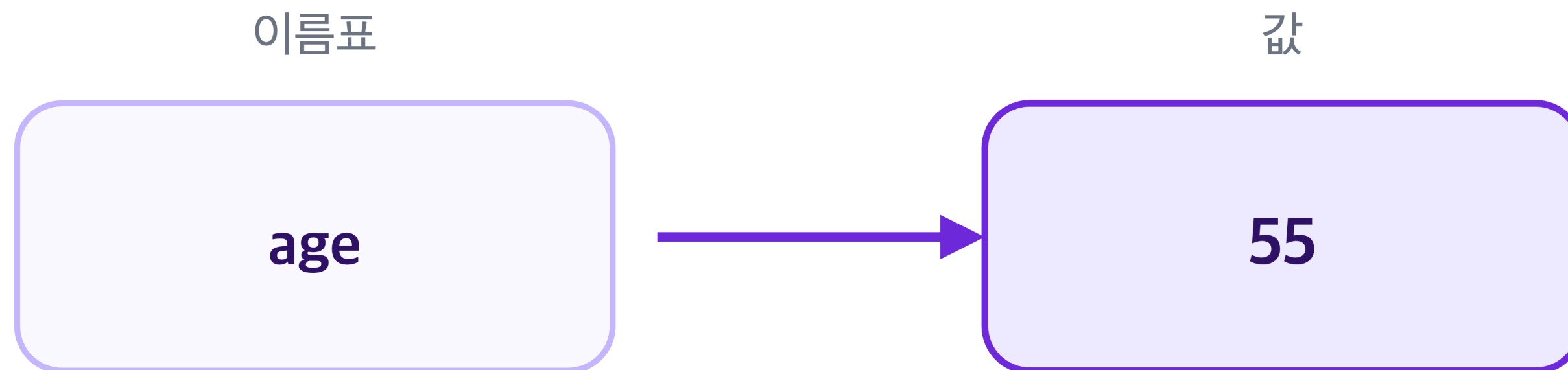


Python 기초



변수란?

- 값을 저장해두는 이름표
- 숫자, 문자, 계산 결과 등을 담는 공간
- 이후 코드에서 다시 사용할 수 있는 값
- 데이터 분석 코드의 기본 구성 요소





변수 이름 작성

- 의미를 알 수 있는 이름 사용
- 공백 대신 언더스코어 사용
- 숫자로 시작하지 않는 이름
- 예약어와 기존 함수 이름 사용 주의

구분	변수명	이유
좋은 예	patient_age	의미 명확
나쁜 예	a / 2age	모호·숫자 시작



숫자형 자료형

- 정수를 표현하는 int
- 소수점을 표현하는 float
- 평균, 합계, 비율 계산에 자주 사용
- 데이터 분석에서 가장 기본이 되는 값 유형



```
age = 55          # int (정수)
height = 1.70     # float (소수)
```




문자열 자료형

- 글자를 표현하는 str
- 이름, 성별, 지역, 진단명 등 텍스트 정보 표현
- 따옴표로 감싸서 작성하는 값
- 숫자처럼 보이지만 문자열일 수 있는 데이터 주의



```
name = "환자A"    # str (문자열)
age  = 55         # int (숫자)
code = "55"       # 숫자처럼 보여도 str
```



불리언 자료형

- 참과 거짓을 나타내는 bool
- True와 False 두 가지 값
- 조건 판단과 필터링에 자주 사용
- 비교 연산의 결과로 생성되는 값

```
In [1]  
age = 65  
print(age > 60)  
print(age < 60)
```



True

False

age > 60 → 참

age < 60 → 거짓



기본 연산

- 숫자 간 사칙연산
- 문자열 연결과 반복
- 비교 연산을 통한 조건 판단
- 데이터 변환과 필터링의 기초가 되는 연산

```
In [1]  
print(3 + 2)      # 산술  
print('a' + 'b')  # 문자열 연결  
print(65 >= 60)   # 비교
```



```
5  
ab  
True
```

Out



자료형 확인과 변환

- type을 통한 자료형 확인
- int, float, str을 이용한 형 변환
- 데이터 파일에서 자주 발생하는 자료형 불일치
- 분석 전 필요한 값의 형태 점검

```
In [1]  
x = "120"          # 문자열  
print(int(x) + 1)  # 숫자로 변환
```



121

Out



리스트란?

- 여러 값을 순서대로 담는 자료형
- 대괄호로 표현하는 값의 모음
- 인덱스를 통해 특정 위치 값 접근
- 여러 컬럼명이나 여러 결과값을 다룰 때 활용

`scores = [90, 85, 77, 60]`

90

85

77

60

[0]

[1]

[2]

[3]



리스트 인덱싱과 슬라이싱

- 0부터 시작하는 인덱스
- 특정 위치의 값 선택
- 범위를 잘라내는 슬라이싱
- 데이터 일부를 선택하는 감각의 기초



`lst[1:3] → [20, 30]`



리스트의 기본 기능

- append를 통한 값 추가
- len을 통한 길이 확인
- in을 통한 포함 여부 확인
- 반복 처리와 데이터 모음 관리에 활용

```
In [1]
lst = [1, 2, 3]
lst.append(4)    # 값 추가
print(lst)      # 추가된 리스트
print(len(lst)) # 길이
print(3 in lst)  # 포함 여부
```



```
[1, 2, 3, 4]
```

```
4
```

```
True
```

Out



딕셔너리란?

- key와 value로 구성된 자료형
- 중괄호로 표현하는 값의 묶음
- key를 이용한 값 접근
- 컬럼명과 값, 설정값, 매핑 정보 표현에 유용

```
In [1]  
patient = {"age": 55, "bp": 138}  
print(patient["bp"])
```



138

Out



딕셔너리 활용 예시

- 환자 정보 표현
- 컬럼명과 설명 매핑
- 라벨 ID와 클래스 이름 매핑
- 분석 설정값 저장



```
patient = {"age":55, "sex":"M", "bp":138}
```



리스트와 딕셔너리 비교

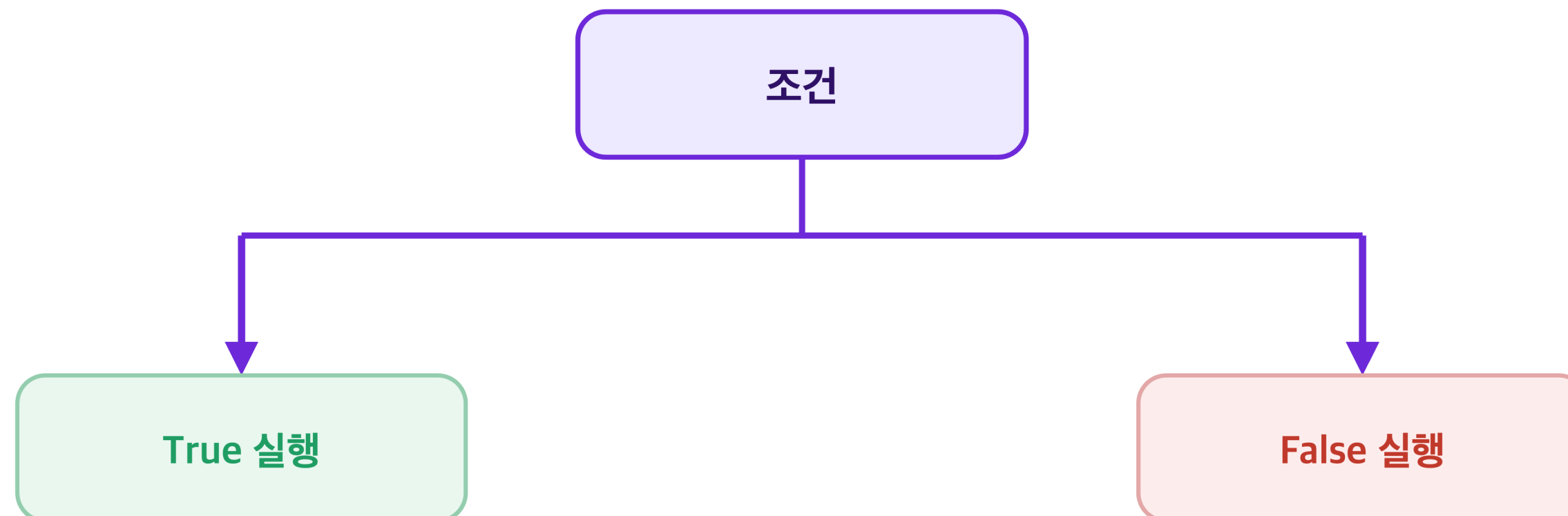
- 순서가 중요한 값 모음인 리스트
- 이름으로 값을 찾는 구조인 딕셔너리
- 여러 샘플을 담을 때 활용 가능한 리스트
- 한 샘플의 속성을 표현할 때 유용한 딕셔너리

자료형	특징	활용
리스트	순서 중요	여러 샘플
딕셔너리	이름으로 접근	한 샘플 속성



조건문이란?

- 조건에 따라 다른 코드를 실행하는 구조
- if, elif, else를 활용한 분기 처리
- True와 False 결과에 따른 실행 흐름
- 데이터 값에 따른 판단 로직 작성의 기초





조건문 예시

- 나이에 따른 그룹 구분
- 검사 수치 기준 정상·주의 판단
- 결측 여부에 따른 처리 분기
- 데이터 변환 규칙 작성의 기본 형태

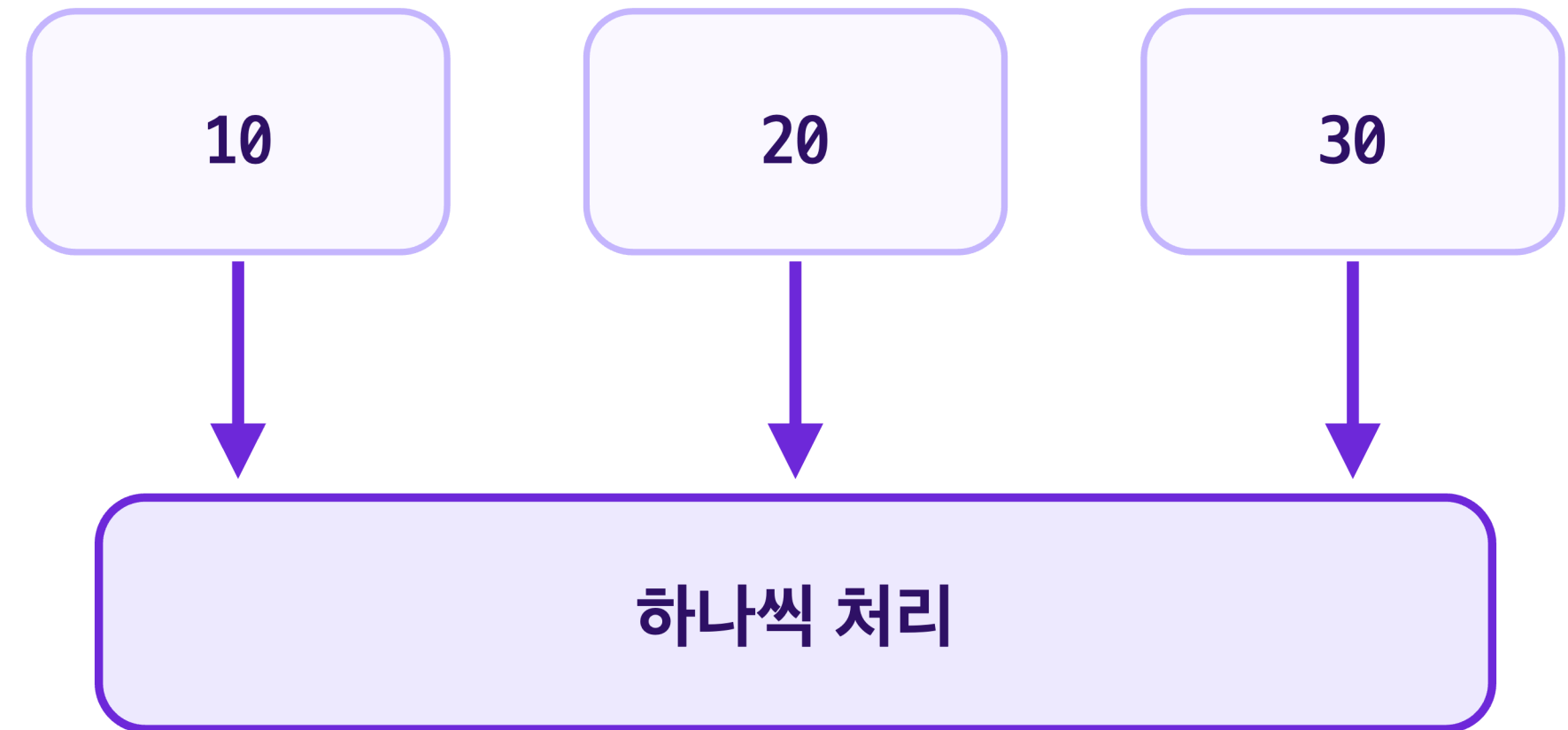
```
if bp >= 140:  
    state = '위험'  
elif bp >= 120:  
    state = '주의'  
else:  
    state = '정상'
```



반복문이란?

- 같은 작업을 여러 번 수행하는 구조
- 리스트의 값을 하나씩 처리하는 for문
- 여러 컬럼이나 여러 파일 처리에 유용한 방식
- 반복 작업을 줄여주는 프로그래밍 기본 도구

```
for x in [10, 20, 30]:
```





반복문 예시

- 여러 숫자의 합계 계산
- 여러 이름을 순서대로 출력
- 여러 컬럼에 같은 처리 적용
- 데이터 분석 자동화의 출발점

```
In [1]  
nums = [10, 20, 30]  
total = 0  
for x in nums:  
    total += x    # 누적  
print(total)
```



60

Out



함수란?

- 자주 쓰는 코드를 이름 붙여 재사용하는 구조
- 입력값을 받아 결과를 반환하는 방식
- 반복되는 처리 과정을 묶는 방법
- 데이터 정제와 분석 코드의 재사용성 향상





함수 사용 예시

- BMI 계산 함수
- 숫자 값을 범주로 바꾸는 함수
- 텍스트 값을 정리하는 함수
- 같은 규칙을 여러 데이터에 적용하는 방식

```
In [1]  
def bmi(h, w):  
    return w / h**2  
  
print(bmi(1.70, 65))
```



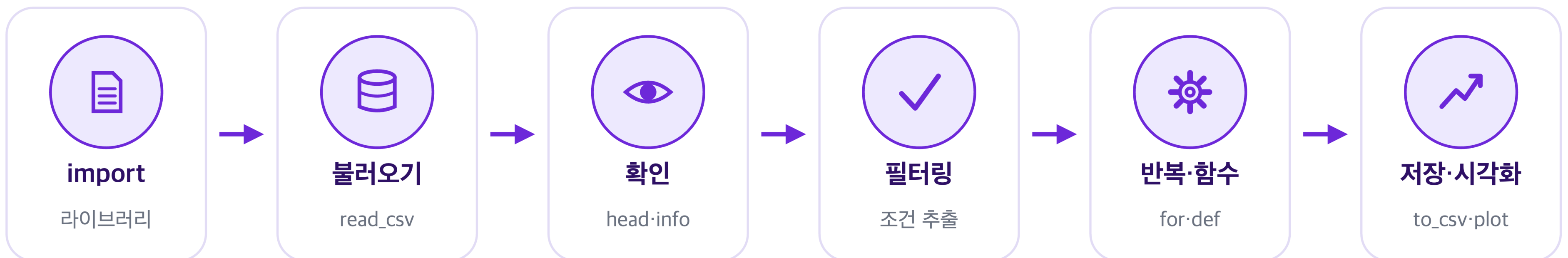
22.5

Out



데이터 분석에서 자주 보는 코드 패턴

- 라이브러리 import
- 데이터 불러오기
- 데이터 확인
- 조건에 따른 필터링
- 반복 처리와 함수 적용
- 결과 저장 또는 시각화





정리

- 데이터 분석을 위한 파이썬 실행 환경 이해
- 값을 저장하고 다루기 위한 변수와 자료형
- 여러 값을 관리하는 리스트와 딕셔너리
- 조건문, 반복문, 함수를 통한 기본 코드 흐름